



Wallet Control

Auditoría técnica, hallazgos de seguridad y plan a versión 1.0

Preparado con Claude Code · claude-workspace
2 de julio de 2026

Contexto

Wallet Control es una app de finanzas personales con asesor de IA integrado ("Finando"), construida en Expo/React Native. Este documento resume la auditoría técnica realizada sobre el código real del proyecto (no solo las notas internas), los hallazgos de seguridad y calidad, y el plan concreto para llevarla a versión 1.0 lista para compartir públicamente.

Diagnóstico general: la app está **funcionalmente completa en un ~85%** — 7 pantallas bien construidas, dark mode completo, exportación de PDF, sistema de categorías, metas y tarjetas maduro. Lo que falta no son más pantallas: es cerrar huecos de seguridad antes de compartir un build real, dejar el proyecto listo para generar un build instalable (hoy no es posible), y ejecutar un lanzamiento controlado.

Hallazgos clave

BLOQUEANTE Seguridad — antes de compartir un build con IA real

1. API key de Anthropic expuesta en el bundle — `lib/claude.ts:91,103` . La llamada a la API se hace directo desde el cliente con la key leída de `EXPO_PUBLIC_ANTHROPIC_API_KEY` . Cualquier variable `EXPO_PUBLIC_*` queda incrustada en el bundle JS de producción — extraíble descompilando el APK. Hoy no hay key real en el repo (la app corre en modo demo), pero es bloqueante para el día que se active la IA real.

2. Contraseñas en texto plano — `lib/storage.ts:78-86` guarda `password: string` sin hash en AsyncStorage; `app/onboarding.tsx` compara en texto plano. `app/ayuda.tsx:327` afirma —incorrectamente— que "tu contraseña es segura".

3. Modelo desactualizado (menor) — `claude-sonnet-4-6` sigue activo, pero `claude-sonnet-5` ofrece mejor relación calidad/costo para coding y trabajo agéntico.

BLOQUEANTE Build y publicación — hoy no se puede generar un APK/IPA

4. Falta `ios.bundleIdentifier` y `android.package` en `app.json` .

5. No existe `eas.json` ni `extra.eas.projectId` — sin perfiles de build configurados.

6. `expo-notifications` se usa activamente pero no está declarado como plugin en `app.json` .

7. Assets rotos: `icon.png` conserva guías de diseño sin limpiar; `splash-icon.png` es la plantilla vacía, sin logo.

CALIDAD Consistencia y pulido — no bloquean, se notan

#	HALLAZGO	ARCHIVO
8	Promesa de "Excel/CSV próximamente" que no existe ni está en desarrollo	app/ayuda.tsx
9	Markdown del chat de Finando se muestra literal (asteriscos visibles)	components/ChatBubble.tsx
10	Componente stub vacío, código muerto	components/QuincenaCard.tsx
11	Componente de presupuesto completo pero nunca montado en ninguna pantalla	components/BudgetProgressBar.tsx
12	Falta catch en el login — error no capturado	app/onboarding.tsx

13	No avisa si se deniega el permiso de notificaciones	components/CardFormModal.tsx
14	Versión desincronizada: perfil muestra v0.6.0, package.json dice 1.0.0	app/(tabs)/perfil.tsx
15	Sin tests automatizados (aceptable para el tamaño actual)	—
16	Sin accessibilityLabel/Role en botones de solo-ícono	toda la app

YA RESUELTO

- Sin secrets hardcodeados ni trackeados en git; .gitignore cubre .env correctamente.
- Manejo de errores de red razonable en lib/claude.ts (fallback sin crash).
- Modo demo bien pensado — permite compartir la app sin costo de API mientras no se active la key real.
- Prompt caching implementado correctamente (cache_control: ephemeral sobre el system prompt).
- Diseño visual, dark mode y arquitectura de pantallas sólidos.

Plan a versión 1.0

Fase 1 — Cerrar bloqueantes (1-2 sesiones)

- ☐ Mover la llamada a Finando IA a un backend/proxy (o mantener el modo demo como modo público mientras no exista proxy)
- ☐ Eliminar o asegurar el campo password (expo-secure-store, o quitarlo si no hay backend real)
- ☐ Agregar bundleIdentifier / package y el plugin de expo-notifications en app.json
- ☐ Correr eas build:configure para generar eas.json
- ☐ Re-exportar íconos y splash sin las guías de diseño
- ☐ Migrar a claude-sonnet-5 cuando se active la IA real

Fase 2 — Pulido de producto (recomendado)

- ☐ Quitar o cumplir la promesa de exportación a Excel/CSV
- ☐ Renderizar Markdown básico en el chat de Finando
- ☐ Integrar BudgetProgressBar a Resumen o eliminarlo; eliminar QuincenaCard
- ☐ Sincronizar la versión mostrada con package.json
- ☐ Agregar el catch faltante en login y el aviso de permisos denegados

Fase 3 — Roadmap de features (priorizado desde NOTAS/ideas wallet-control.txt)

Después de publicar 1.0, con feedback real primero: notificaciones de presupuesto, exportar Excel/CSV, búsqueda de gastos, comparativa mes a mes. Para v2.0 o con tracción: modo pareja/familia, categorización automática por IA, integración bancaria.

Cómo empezar a compartirla y conseguir audiencia

1 Build compatible sin fricción. Con la Fase 1 lista, correr eas build --platform android --profile preview genera un APK instalable directo, compartible por link a un grupo cerrado de confianza (5-10 personas).

2 Beta cerrada (2-3 semanas). Recoger feedback específico (¿usan Finando IA?, ¿entienden el modo anónimo?, ¿qué confunde del onboarding?) antes de exponerse a audiencia más grande. En paralelo, subir a Google Play

Internal Testing (gratis).

3 Fachada pública en paralelo. Landing de una sola página con la skill PAKI: qué hace la app, 2-3 screenshots reales, botón de descarga o lista de espera.

4 Lanzamiento público. Play Store (cuenta de desarrollador, \$25 único pago) + Product Hunt + comunidades de finanzas personales en español (subreddits, grupos de Telegram/Facebook, comunidades locales de Colombia).

5 Ciclo de audiencia continuo. Usar el historial de ACTUALIZACIONES.txt como base para publicar novedades en redes cada vez que se agregue algo del roadmap — mantiene interés y da razones para volver a abrir la app.

Verificación

- **Fase 1 completa cuando:** el build preview de EAS corre sin errores y genera un APK instalable, sin la API key visible en el bundle.
- **Fase 2:** probar el flujo completo — onboarding → registrar gasto por chat → ver categoría en Resumen → exportar PDF — en el APK generado, no solo en Expo Go.
- **Fase 4:** confirmar que al menos 3-5 personas fuera del creador instalaron y usaron la app desde el APK compartido antes de subir a Play Store.